

Laboratory 1 - Build **svcdesk** from a contradictory specification

AI Assessment Scale level 4 (Full AI) · peak container RAM about 0.4 GB · the Core items sum to 110 minutes, leaving 40 minutes of slack.

Read these first, in this order. All four are in *Laboratory 1 package* in this topic.

1. **REQUIREMENTS.md** - what the service desk wants, requirements R-01 to R-25. This is the document that holds the three contradictory pairs.
2. **API.md** - the exact HTTP contract the checker enforces, with the SLA test vectors.
3. **CHECKS.md** - every check the checker runs, with what it expects. The admissible outcomes of each contradiction are visible here, which is why the pairs are discoverable by reading rather than by guessing.
4. **DECISIONS-template.md** - the shape of your reasoning artifact. The same skeleton already sits in your repository as **DECISIONS.md**.

HANDOUT.md, in the same folder, is the full laboratory sheet: the minute budget for each Core item and the checklist to run before you submit. `./itsmlab.sh checks 1` prints the same catalogue as **CHECKS.md** straight from the checker, and `--markdown` prints it in exactly that format; it runs nothing, so it is safe at any point in the laboratory.

You build the service-desk API that the whole semester runs on, using spec-driven development (GitHub spec-kit driven by Claude Code, by Gemini CLI or GitHub Copilot Free, or written by hand), Python 3.13 with FastAPI or Go, Docker Compose and pytest.

The supplied requirements document contains **three pairs of requirements that cannot both hold**: C1 (the SLA clock for P1), C2 (closed tickets and reopening) and C3 (VIP reporters and the priority matrix). Each conflict has exactly two admissible resolutions, and the conformance suite accepts any of the eight combinations - provided **DECISIONS.md** declares the resolution your running service actually exhibits, and defends it. A single prompt produces code; it cannot produce a decision about which requirement to break and why. That decision, defended in writing, is the work of this laboratory.

Core specifications (all must pass; there is no partial credit inside the bundle):

- **L1-CORE-1 compose-up** - the compose contract: a service **svcdesk** built from your repository (a **build:** key, never **image:** alone), no bind mounts in the resolved configuration, **docker compose up --wait** and **/health** answering within 120 s.
- **L1-CORE-2 conformance** - 49 HTTP checks: health, validation, the priority matrix, the state machine, the reopen window, the SLA vectors, breach and pause, the test clock.
- **L1-CORE-3 decisions-structure** - **DECISIONS.md** has the front matter, the admissible values, three sections, and five labels of at least 20 characters each.
- **L1-CORE-4 decisions-consistency** - the values declared in **DECISIONS.md** equal the ones the checker observed on your running service.
- **L1-CORE-5 spec-first** - grader-only: a **specs** receipt exists whose commit is an ancestor of the submission, and every commit adding a file under **src/** descends from it. Locally this spec is always reported as **skip**.

Stretch: any two of three options; each lifts the grade band equally. One of them is a **/speckit.converge** validation report; another is an agent configuration with a narrow **disallowedTools** denylist plus an **AGENT-POLICY.md** that justifies each denial as a blast-radius decision.

Rules. No network at run time: your image installs its dependencies at build time. Build, do not pull. No bind mounts, in any override file or through `.env`. Specs before code. Tags never move: each attempt is a new tag (`lab1/v1`, `lab1/v2`, `lab1/v3`), and a tag that moves after its receipt voids that attempt while still consuming it.

Deliverable. A public GitHub repository created from the course template, at the receipted tag, containing `specs/`, `src/`, `DECISIONS.md`, `docker-compose.yml` and a passing conformance run. The grader runs the checker itself; your reported result is not used.



Laboratory 1 package

The five documents of Laboratory 1, exactly as the course team wrote them. Read them in this order: `REQUIREMENTS.md`, `API.md`, `CHECKS.md`, `DECISIONS-template.md`. `HANDOUT.md` is the full laboratory sheet: the minute budget, the spec-kit chain with the exact commands, the Stretch options and the checklist to run before you submit.

These files are the source of truth for what the checker enforces. Where this Moodle page and these documents differ, the documents win.

Laboratory instructions

Every command runs in the root of your repository. On Linux and macOS use `./itsmlab.sh`; on Windows PowerShell use `.\itsmlab.ps1`, and `;` where the examples chain commands with `&&`. The order below matters: the specification is receipted *before* any code exists, and `L1-CORE-5` checks exactly that.

Step 0. Check `itsmlab.yaml`

Before your first `submit`, `repository:` must hold your real `<your GitHub login>/<your repository>`. The template's placeholder is refused with a message. The same file holds `lab:`, `baselines:` (for each earlier laboratory, whether you are building on your own accepted submission or on the published checkpoint), `submissions_repo:` and `checker_image:`.

Step 1. Write the specification, with nothing under `src/` yet

What you write it from: `REQUIREMENTS.md` says what the service desk wants, `API.md` gives the exact HTTP contract your service has to honour, and `CHECKS.md` lists every check with its expected value, so you can see before you write a line which resolutions of the three contradictions the conformance suite will accept. All three are in *Laboratory 1 package* in this topic. Your specification is your reading of those documents: what the service will do, including which side of each contradiction it takes.

The tree must contain at least one file of 500 bytes or more under `specs/` (`.gitkeep` and `README.md` do not count) and no file under `src/` other than `src/README.md`. Then commit and push:

```
git add -A
git commit -m "Lab 1 specs"
git push origin main
```

Step 2. Take the specs receipt

```
./itsmlab.sh submit 1 --kind specs
```

The command never opens an issue for you. It checks locally what it can (that **repository:** is filled in, and that the commit it will name is already on **origin**), then prints a prefilled issue-form URL. Open it in a browser while logged in to GitHub as the login on the roster, check the fields, and click **Submit new issue**. Within about a minute the course bot comments with your receipt and labels the issue **receipted**. **Only then start writing code.**

If the bot refuses instead (the repository is not on the roster, the tag was not found, this would be a fourth attempt), fix the cause and open a new issue: a refused issue is not an attempt.

Step 3. Implement, and run the checker as often as you like

The request and response shapes, the status codes and the SLA vectors your service must match are in **API.md**; the check identifiers the table prints back at you are listed in **CHECKS.md**.

```
./itsmlab.sh verify 1
./itsmlab.sh verify 1 --json report.json
```

Local runs are unlimited and never count as attempts. The exit code is **0** when every Core specification passes, **1** when at least one does not, and **2** when the checker itself could not run. The table prints one row per specification, with status **pass**, **fail**, **error** or **skip**; **L1-CORE-5** is always **skip** locally, because only the grader can see your receipts. Core passes when no Core specification is **fail** or **error**.

On the unmodified template, expect the build to fail: **L1-CORE-1** fails with the compose error in its message, **L1-CORE-2** is reported as **error** because the service never started, and **L1-CORE-3** fails on the template's **TODO** placeholders. That is the normal starting point, not a broken setup.

Step 4. Commit and push the implementation

```
git status
git add -A
git commit -m "Lab 1 attempt 1"
git push origin main
```

Step 5. Verify the committed tree

Run **./itsmlab.sh verify 1** once more and check two things: it exits 0, and its **commit** line does **not** say (**dirty**). (**dirty**) means your working directory holds changes that are not in the commit, so the grader would evaluate different files from the ones you just tested.

Step 6. Tag that commit and push the tag

```
git tag -a lab1/v1 -m "Lab 1 attempt 1"
git push origin lab1/v1
```

-a creates an annotated tag, which is the form the receipt workflow resolves. Use **lab1/v1** for the first attempt, **lab1/v2** for the second and **lab1/v3** for the third. **A tag never moves**. If you change anything after the receipt, do not move the tag: go back to step 4 and use the next tag name. A tag that moved after its receipt voids that attempt, and the attempt still counts.

Step 7. Take the submission receipt

```
./itsmlab.sh submit 1 --kind submission --tag lab1/v1
```

It prints a **warning**: when the working tree has uncommitted changes or when **HEAD** has moved past the tag, and it prints the commit and the **tree_sha** it will claim. Open the printed URL, check the fields, submit the issue, and keep the receipt comment. Then post that issue URL in the Laboratory 1 assignment below.

Step 8. Read your grade

The grade arrives as a comment **itsmlab grade** on the same issue, usually within twenty minutes, with **grade.json** and its Sigstore bundle. If nothing appears within an hour, ask on the forum with the issue number. How to check that the signature is genuine is on the *How grading works, technically* page in the General section.

What the commands and their parameters mean

Verified against the checker's own command-line definition, not only the handout.

Command or flag	What it does
--version	prints the checker version and exits
--debug	shows pytest's own output on stderr. It is a top-level flag, so it goes before the subcommand: ./itsmlab.sh --debug verify 1 . The environment variable ITSMLAB_DEBUG does the same
doctor	preflight: Docker, compose, git, free disk and repository layout. This is the table you paste into the pre-work assignment
checks 1	prints the published checks of laboratory 1 as a table, and checks 1 - -markdown prints them in the format of CHECKS.md . Nothing is executed, so you can read every rule without starting Docker
verify 1	runs Tier A for laboratory 1 in the current directory
submit 1 --kind specs	prints the prefilled issue-form URL for the specification receipt, using the current main HEAD
submit 1 --kind submission --tag lab1/v1	prints the prefilled issue-form URL for an attempt, naming a tag that must exist locally and on origin
submit N --kind prediction --text "..."	binds a text to the current main SHA. Not used in laboratory 1; it belongs to a laboratory 2 Stretch item. --kind is required and accepts exactly submission , specs or prediction
--json report.json	writes the JSON report to that path (the file is in the template's .gitignore)
--keep	does not tear the compose project down after the run, so you can inspect it
--project NAME	overrides the compose project name; the default is itsmlab- plus 8 hexadecimal characters derived from your directory path

Command or flag	What it does
<code>--prune-images</code>	after the run, also removes the images compose built for this project (<code>down --rmi local</code>); later runs then rebuild from the build cache
<code>--base-url URL</code>	checks a service you started yourself instead of starting it with compose; the file checks still run on the current directory
<code>--probe-window SECONDS</code>	with <code>--base-url</code> : how long to wait for <code>GET /health</code> to answer 200 (10 seconds by default)
<code>--tests-result PATH</code>	with <code>--base-url</code> : the captured result of the <code>tests</code> service, since that service is not run in this mode
<code>ITSMLAB_IMAGE</code>	environment variable choosing the checker image. Order of precedence: <code>ITSMLAB_IMAGE</code> , then <code>checker_image</code> in <code>itsmlab.yaml</code> , then <code>ghcr.io/swasik/itsmlab:2026</code>

The wrapper passes every argument through to the checker unchanged. It mounts your current directory at the same absolute path on Linux and macOS, and at `/work` on Windows, and it mounts the Docker socket (`/var/run/docker.sock`, unless `DOCKER_HOST` points at another Unix socket, as with rootless Docker, Colima or OrbStack).

Under `--base-url` the URL must be reachable from inside the checker's own container: on Docker Desktop that is `http://host.docker.internal:8080`, on Linux `http://172.17.0.1:8080`.

What the receipt fields mean

Three fields decide how an attempt counts. `attempt` is 1, 2 or 3, and the best of them counts; a fourth issue is refused. `after_attempt1_due: true` means the receipt arrived after the advisory Sunday deadline: it is recorded and **still counts**. `late: true` means it arrived after the correction window closed at the start of the next session: it is graded and **does not count**. `commit`, `tree_sha` and `archive_sha256` are the content digest Tier B grades; nothing outside that digest is evaluated.

Deadlines. Attempt 1 is due **Sunday 20 September 2026, 23:59:59** (Europe/Warsaw); this deadline is advisory, a later receipt is marked and still counts. Attempts 2 and 3 are the correction window and must be receipted before **Saturday 26 September 2026, 08:00:00**, when session 2 starts; a receipt after that instant is late and does not count. The best of the up to three attempts counts.

Every file under `src/` and `specs/` with a code or Markdown extension, plus `DECISIONS.md`, must carry an **AI-disclosure header** in its first ten lines, matching `ai-generated: <0-100>% - <one line on how>`. Missing headers are reported as an advisory in `report.json`, not graded.



Laboratory 1 - svcdesk from a contradictory specification



Laboratory 1 - Build `svcdesk` from a contradictory specification.

Your work is submitted in GitHub, through a receipt. This Moodle activity records it, so that the mark has a place in the gradebook and so that you can see at a glance what you handed in. Nothing you paste here is graded; the grader evaluates the receipted digest.

Paste here, as online text:

1. the URL of your **submission issue** in `swasik/itsm-2026-submissions`, in the form `https://github.com/swasik/itsm-2026-submissions/issues/<number>`. It is the issue you opened from the URL that `./itsmlab.sh submit 1 --kind submission --tag lab1/v1` printed, and it must carry the labels `receipted`, `lab:1` and `kind:submission`. If it does not carry `receipted`, the bot has not accepted it and it is not a submission;
2. the **tag** you submitted: `lab1/v1`, `lab1/v2` or `lab1/v3`;
3. the URL of your **specs receipt issue** (labels `receipted`, `lab:1`, `kind:specs`). That receipt is the evidence for `L1-CORE-5`, the specification-before-code rule, which is the one Core specification your local checker can never decide.

If you file a later attempt, edit this submission and paste the newest issue URL and tag.

What is actually graded. Tier B clones your public repository at the receipted tag and grades exactly those files. Your working directory, your local checker output and the text in this box are not evaluated; the grader runs the checker itself. A tag that moved after its receipt voids that attempt, and the attempt still counts.

Deliverable in the repository: `specs/` (receipted before any code), `src/`, `DECISIONS.md`, `docker-compose.yml`, and a passing conformance run.

Weight: 12.5 % of the course - Core bundle 8 %, Stretch bundle 3 %, reasoning artifact (`DECISIONS.md`) 1.5 %.

Attempt 1 due Sunday 20 September 2026, 23:59:59 (advisory - a later receipt is marked `after_attempt1_due` and still counts). **Attempts 2 and 3 must be receipted before Saturday 26 September 2026, 08:00:00**; after that instant a receipt is marked `late`, is graded and does not count. The best of the up to three attempts counts. Local checker runs are unlimited and never count.

Step-by-step instructions, with every parameter explained, are in *Laboratory instructions* above. The machinery behind the grade is described on the *How grading works, technically* page in the General section.